Post-Quantum Access Tokens for CDNs: Pooled Batching Relieves the Issuer, Requests per Token Decide the Edge

Dhavan Arjampudi

***Abstract*—CDN signed URLs and cookies use either HMAC keys shared with every edge, which are post-quantum but let a compromised edge mint tokens, or RSA, ECDSA and Ed25519 keys, which quantum computers would break. We compare post-quantum tokens with 13 baselines using microbenchmarks, a Go edge prototype, real HTTP/2 clients, a fleet simulator and Meta's public CDN traces. Two designs remain: M2 sends a per-token ML-DSA-44 signature once per connection (stapling), and EMBER, a CDN profile of MTL-mode batch signing, signs one Merkle root per 10 ms for a pooled issuer key. *At the issuer, busy pools win*: EMBER cuts issuer CPU 19–144-fold in simulation and, at the full rates implied by the trace, needs 0.38–1.08 cores against 177 for M2. *At the edge, the winner depends on the requests K a token carries at one verifier.* With session tokens ($K \approx 150$), M2 matches HMAC's edge CPU in two prototype campaigns (25.35 vs. 25.90 µs per request). With one signed-URL token per object per hour, our assumption for the trace ($K = 1.7$–2.4), M2 costs 53.8 against 1.9 µs and $1{,}869$ against 426 B per request for pooled EMBER. From measured costs, M2 wins only above $K \approx 48$–154 (CPU) and $K \approx 9$–27 (bytes). On the prototype, a lost cache entry costs about 36 ms, a restart a 258–263 ms reconnect, and an undersized cache makes M2 retry on 55% of requests. We derive deployment rules keyed to K, the per-key rate and issuer-key ownership.**

***Index Terms*—Access tokens, content delivery networks, CDN traces, Merkle trees, ML-DSA, post-quantum cryptography, signed URLs.**

I. Introduction

CONTENT delivery networks (CDNs) protect paywalled video, software downloads and other private objects with *access tokens*. An issuer, usually the content owner's login service, gives an authorized client a short-lived token. The token binds a URL prefix, an expiry time and optional client attributes. The client attaches it to every request: as a query parameter (a *signed URL*), as a cookie, or in a header. Any edge server can check the token before serving from cache, without contacting the origin. How many requests one token carries varies widely. A token scoped to a streaming session covers one manifest request and then hundreds of segment requests. A signed URL for a single object may be used only once or twice.

Deployed schemes use one of two approaches. Symmetric schemes such as Akamai's EdgeAuth [1] append an HMAC [2] under a key held by every edge. Public-key schemes sign the token: CloudFront key groups accept RSA-2048 or ECDSA P-256 keys [3], and Google Media CDN uses Ed25519 [4]. HMAC-SHA256 is not threatened by quantum computers, but every verifier can also mint tokens. CDNs already hold many customers' secrets [5], [6], so a single compromised edge exposes every path the key covers. The public-key schemes keep edges secret-free, but Shor's algorithm [7] would recover their signing keys from long-lived public keys. Authentication has no harvest-now-decrypt-later exposure. The reason to migrate now is lead time: token formats, SDKs and edge verifiers must change before a cryptographically relevant quantum computer (CRQC) exists, and industry roadmaps target post-quantum authentication by about 2029 [8].

The obvious replacement is one FIPS 204 ML-DSA signature per token [9], [10]. It produces a 3,350-B token, which every request then repeats. Two remedies are available. *Stapling* detaches the signature and sends it once per connection (M2). *Merkle batch signing* [11]–[13] signs one root for many tokens, which our earlier version of this work applied to CDN tokens as EMBER. Review of that version showed that at the operating point we had recommended (per-tenant trees, 10 ms epochs) most trees hold a single token and most of the edge benefit came from detaching the signature. A replay of Meta's public CDN traces [14], [15] then showed that even this edge verdict holds only when tokens carry many requests. This paper separates the two mechanisms, measures each against strong alternatives, and identifies what decides between them at the edge: the number of requests K a token carries at one verifier.

Contributions.

1) **A trust and threat comparison** of 13 baselines and M2 (Section III, Table VIII). It covers who can mint tokens after an edge compromise, the post-quantum assumption and token size; HMAC is already post-quantum secure.
2) **Pooled batching relieves the issuer** (Sections VI-C and VI-D). EMBER, a CDN profile of MTL-mode and batch signatures rather than a new primitive, pays off when one tree is pooled per issuer key, which the pooled tenants must share, and the pool exceeds about 10^3 tokens/s at $\Delta = 10$ ms. It cuts issuer CPU 19–144× in simulation. At the full rates implied by the trace, per-content-type trees hold a median of 547 tokens and the issuer needs 0.38–1.08 cores against 177 for per-token signing; the single-token trees we reported earlier appear only at low rates.
3) **Requests per token decide the edge** (Sections VI-B and VI-D). With session tokens ($K \approx 150$), M2 matches

D. Arjampudi is with Andhra University, Visakhapatnam, India (e-mail: *[editable]*).

[Editable: funding statement, ORCID, corresponding-author note.]

HMAC in whole-request edge CPU in a Go prototype and is the smallest post-quantum option on the wire. With one token per object per hour ($K = 1.7$–2.4 on the trace), M2 verifies a signature about every second request and pooled EMBER is $28\times$ cheaper in edge crypto and $4.4\times$ smaller on the wire. From measured costs we derive the crossover: $K^* \approx 48$–154 for CPU and 9–27 for bytes.

4) **HPACK behavior of real clients** (Section VI-E). Chrome 153 adopts an edge-advertised 64 KiB HPACK table, so a 3.3 KB ML-DSA cookie costs 25.7 B per request; curl/nghttp2 and Go clamp it to 4096 B, yet Go kept the cookie indexed (228 B per request) while curl re-sent it.
5) **A prototype and a simulator.** The simulator matches the prototype on miss and byte counts in all 12 configurations, and we report where it fails (Section VI-H). The prototype runs M2 with its retry path and pooled EMBER for edge CPU, latency under load, and recovery after cache loss, restarts and undersized caches (Sections IV and VI-B).
6) **Deployment guidance** keyed to K, the per-key token rate and who holds the issuer key (Section VII), including when delegated Ed25519 keys suffice: only if a key cannot be broken within its exposure window, for which published estimates range from 38 h to 10 min per key.

II. Background and Related Work

A. CDN Access Tokens and Header Compression

A CDN token consists of *claims* and an *authenticator*. Typical claims are a URL prefix, an expiry time and optionally a client IP range. Media CDN appends `Signature=` and an unpadded base64url Ed25519 signature [4]. EdgeAuth appends `hmac=` [1]. Size limits depend on where the token is carried. RFC 9110 recommends that URIs of at least 8000 octets be supported [16]. Some legacy clients accepted only about 2 KB, for example Internet Explorer's limit of 2,083 characters [17]. RFC 6265 requires at least 4096 B per cookie [18]. Per-token ML-DSA-44 tokens fit the RFC limits but not the legacy one.

In HTTP/2, request headers are compressed with HPACK [19]. A repeated header field, such as a cookie, is sent as a short index once it sits in the dynamic table. Two parties set the size of that table. The decoder, which for requests is the edge, advertises a maximum with SETTINGS_HEADER_TABLE_SIZE [20]. The encoder, which is the client, may use any smaller size and signals its choice with a Dynamic Table Size Update [19]. Whether a 3.3 KB cookie stays indexed therefore depends on the client as well as the CDN (Section VI-E).

B. Post-Quantum Signatures and Their Cost

FIPS 204 specifies ML-DSA, whose ML-DSA-44/65/87 signatures are 2420, 3309 and 4627 B [9]. FIPS 205 specifies SLH-DSA, with 7856 B for 128s and 17,088 B for 128f [21]. RFC 9964 registers ML-DSA for JOSE and COSE and cautions that ML-DSA may not suit settings that need small signatures [10]. Post-quantum TLS measurements find that signature *size*, more than CPU, drives the cost [22]–[25]. CDNs already use ML-KEM key exchange at the edge and toward origins [26], [27], and at least one accepts ML-DSA certificates for origin authentication [28]. KEMTLS removes handshake signatures [29], [30], but tokens are verified offline, outside any handshake. We combine hybrid signatures with the AND-combiner analyzed by Bindel et al. [31].

C. Merkle Batch Signing and Signed Tree Heads

Merkle's tree authentication [11] underlies all of the following schemes. Aguilar-Melchor et al. formalize *batch signatures*, in which one signature covers a tree of messages, and they evaluate them for TLS [12]; a TLS batch-signing draft [32] follows the same idea. *MTL mode* [13], [33] signs ladders over a message series. Each message is then authenticated by a *condensed* signature, a path to a ladder rung. A verifier that lacks the rung needs the *full* signature, which includes the signed ladder. The DNSSEC profiles ML-DSA-MTL [34] and SLH-DSA-MTL [35] apply this pattern. ML-DSA-MTL uses the same ML-DSA-44 root and condensed-path structure as EMBER. Merkle Tree Certificates (MTC) [36], [37] batch certificate issuance and ship selected signed tree heads, called landmarks, to relying parties ahead of time, and a CDN operator is building a CA on them [38]. MTAC profiles MTC for AI-agent identity, not runtime authorization [39]. SLH-DSA's tweakable-hash framework [40] supplies our node domain separation.

EMBER's token is an MTL condensed signature over a per-epoch series, and its root stapling is MTL's full signature sent once per edge. Its pull and push modes correspond to MTC landmark distribution. We found no work, up to November 2026, that applies these mechanisms to CDN signed URLs or cookies. This paper adds not a mechanism but a measurement of where each pays off in a CDN, and where it does not. CacheLib's public traces [14], [15] let us test this on real CDN request streams.

D. Other Token Designs

Delegated credentials [41] let a long-term key certify short-lived keys (our B11). Macaroons [42] attenuate HMAC bearer tokens but keep HMAC's shared-secret trust, and DPoP [43] binds a token to a client key, which is orthogonal to how it is authenticated. Post-quantum Privacy Pass [44] provides unlinkable tokens of 85–175 KB, for a different goal. Sharing keys across servers is known to enlarge the damage of a single compromise [6], [45].

III. System and Threat Model

Entities. An *issuer* I holds a signing key sk with key id kid. The issuer is the tenant's login service or an issuer that the CDN operates for its tenants. *Edges* hold pk, a cache of verified tokens or roots, and a revocation set. In the batched design, a *root service* publishes signed roots as immutable, cacheable objects behind the CDN's shield tier. *Clients* obtain a token and then send a manifest request and K segment requests, normally to one edge.

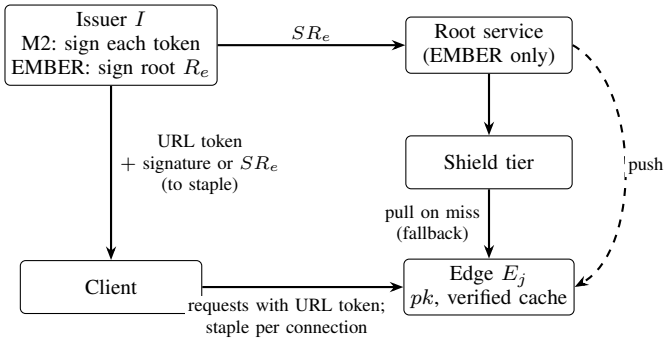


Fig. 1. Stapled post-quantum tokens. In M2 the client staples its token's ML-DSA signature on the first request of each connection. In EMBER it staples the signed epoch root SR_e (MTL's "full" signature) and the URL carries a Merkle path. Pull from the root service through the shield tier and push are EMBER fallbacks.

Why not simply HMAC? HMAC-SHA256 tokens are post-quantum secure. Their weakness is that every verifier can also mint tokens. One compromised edge, whether through memory disclosure, an insider or partner hardware, yields a key that mints tokens for every edge and every path the key covers. A multi-CDN content owner must give each vendor the minting key, and there is no issuer accountability. Public-key tokens give edges only pk. The benefit is limited: a compromised edge can always serve whatever it caches without checking tokens. Asymmetric tokens therefore remove only *lateral* minting, that is, tokens accepted by other edges, other CDNs or other tenants. Table VIII gives this blast radius for each scheme.

Adversary. The adversary $\mathcal{A}$ (i) controls the client–edge network and can replay tokens it observes; (ii) may compromise any subset of edges and shields, including their caches; (iii) can obtain tokens for claims of its choice as a legitimate client, while salts and nonces stay issuer-chosen; and (iv) is quantum. It can run Shor's algorithm against any public key and generic quantum search against hash functions. $\mathcal{A}$ does not obtain sk. An issuer compromise is handled by rotating kid. Theft and replay of bearer tokens are unchanged from the status quo and are mitigated as they are today, by IP or ASN claims, short validity, or DPoP-style binding [43].

Goals. **G1**: an honest edge accepts no claims that I did not issue or that I revoked. **G2**: edges hold no minting secret. **G3**: validity is bounded; for EMBER, the expiry is at most $T_{\text{start}}(e) + \Delta_e + V_{\max}$ plus a skew tolerance τ. **G4**: an authentication path reveals nothing about other clients' claims. **G5**: an edge that cannot verify a token fails closed. The operational requirements are small tokens, per-request edge cost close to HMAC, bounded issuance delay and issuer cost, revocation, and dissemination traffic that is negligible next to the content served.

IV. Design Space

All schemes authenticate the same canonical claim string, so only the authenticator and its delivery differ. Fig. 1 shows the two post-quantum designs this paper focuses on.

A. Per-Token Signatures, Cached or Stapled

B5/B6 put an ML-DSA-44/65 signature in every token. **B5v** adds an edge cache of verified tokens, keyed by SHA-256 of the token, so verification runs once per (token, edge). The 3.3 KB token is still sent and hashed on every request. **M2** (stapled per-token ML-DSA-44) separates the signature from the token. The URL token is the claims followed by `:S=`b64url($kid \parallel$ 16-B nonce). On the first request of each connection, the client sends the 2,420-B signature over (claims, nonce) in a header. We use the connection as the staple unit because a client can observe connections but not edges. The edge verifies the signature, caches $H(ref \parallel \text{claims})$ as verified until the token's expiry, and from then on hashes about 138 B per request.

Recovery. M2 has no pull fallback. An edge that holds no valid entry for a token answers with 401 and a `Staple-Required` hint, and the client retries the same request with the signature header. This happens after a cache eviction, an edge process restart, or a server that does not share the cache. The R2 prototype implements it as a 401 with a `Staple-Required` header (the R1 prototype used a 403). Its cost depends on what was lost (prototype P4, Section V). After a cache flush on a live connection, the retry adds 35.8 ms on average (p99 62 ms), about 1.8 times the 20.3 ms base round trip, plus a 3,242-B header line and one verification. After an edge restart, every client reconnects, at 263 ms on average (p99 427 ms), about 13 round trips; pooled EMBER pays the same (258 ms). One flush or restart per 30-s run caused 4.4–6.4 recoveries per 1,000 requests, and no request failed. An undersized cache is worse: with 64 entries for about 6,000 tokens per run, M2 retried on 55% of requests and its median latency doubled to 41.7 ms, whereas pooled EMBER, which needs one entry per tree, retried on 10%. With 256 entries neither retried. The edge state is small. In the prototype an entry is a 16-B key plus an 8-B expiry (24 B, excluding hash-table overhead), and it lives at most until the token expires ($\le V_{\max} = 1$ h). At the busiest simulated edge, new (token, edge) pairs arrive at 165.8/s ($\bar{\lambda} = 2000$/s) and 1,658/s (20,000/s). This bounds the live set at 0.60 and 5.97 million entries, or 14.3 and 143 MB of payload. An edge that retains this state, and is sized for it, needs recovery only after a restart. How often edges flush or restart in production we cannot measure. M2 is the $N = 1$ special case of EMBER, and it is also B5v without the per-request resend. Its security is the EUF-CMA security of ML-DSA plus the collision resistance of the cache key.

B. EMBER: Pooled Epoch-Merkle Batching

Notation. H is SHA-256 truncated to $n \in \{16, 32\}$ bytes. The tweakable hash is $\mathsf{Th}(T, x) = \text{trunc}_n(H(T \| x))$ with a fixed-length 26-B tweak $T = \texttt{"EMBERv1\0"} \| kid \| e \| type \| \ell \| j \| tenant$, so every node position of every epoch is a distinct function [40]. Leaf i of epoch e is $L_i = \mathsf{Th}(T_{\text{LEAF}}(e, i), r_i \| \text{enc}(c_i))$, where r_i is a 128-bit salt that the issuer draws *after* it receives the claims c_i. Inner nodes are $\mathsf{Th}(T_{\text{NODE}}(e, \ell, j), \text{left} \| \text{right})$, and the tree has the RFC 9162

Algorithm 1 Edge verification of an EMBER token

procedure VERIFY(tok, url, now)
  parse claims c and $(kid, e, i, r, \text{path})$; reject if malformed or kid unknown
  reject if $now > c.Exp + \tau$ or $c.Exp > T_{\text{start}}(e) + \Delta_e + V_{\max} + \tau$
  reject if URL prefix or IP binding fails, (e, i) revoked, or $i \geq N_e$
  $R \leftarrow RootCache[kid, e]$
  if $R = \perp$ **then**
    if a stapled SR_e is present **then** verify σ_e (per-client rate limit); cache
    else fetch SR_e via shield (coalesced, negative-cached); verify; cache
    end if
    on failure or timeout: **return** reject ▷ fail closed
  end if
  $x \leftarrow \mathsf{Th}(T_{\text{LEAF}}(e, i), r \| \text{enc}(c))$
  for each sibling s_ℓ in path **do** $x \leftarrow \mathsf{Th}(T_{\text{NODE}}(e, \ell, \cdot), \text{order}(x, s_\ell))$
  end for
  return $x \stackrel{\text{ct}}{=} R$
end procedure

shape [46], with depth $d = \lceil \log_2 N_e \rceil$. The signed epoch root is $SR_e = (kid, e, T_{\text{start}}, \Delta_e, N_e, R_e, \sigma_e)$, where $\sigma_e = \mathsf{Sig}(sk, \texttt{"EMBER-root"} \| kid \| e \| T_{\text{start}} \| \Delta_e \| N_e \| R_e)$ is ML-DSA-44 with the FIPS 204 context string `EMBER-root`. In the hybrid variant, σ_e is an Ed25519 signature and an ML-DSA-44 signature over the same message, and both must verify.

Token. The URL token is `URLPrefix=...:Expires=...:Tenant=...:E=`*blob*, where the blob is the base64url encoding of version, algorithm, kid, e, i, r_i and the path, $30 + nd$ raw bytes in all. With c_{enc} bytes of encoded claims,

$$S_{\text{tok}} = c_{\text{enc}} + 3 + \lceil 4(30 + nd)/3 \rceil. \quad (1)$$

Our encoder implements Eq. (1). With a 60-character path ($c_{\text{enc}} = 118$) it yields 161, 268, 332 and 460 B for $N = 1$, 2^5, 2^8 and 2^{14}, against 148 B for M2.

Issuance and pooling. The issuer collects requests for Δ seconds, or until a count cap $N_{\max}$ is reached, and then closes the epoch. Closing builds the tree ($2N - 1$ hashes), signs the root once, and returns to each client its token and SR_e. Before the epoch or index counters would wrap, the issuer rotates kid. The unit of batching is the *pool*, meaning the set of tenants that share a tree. Batching saves signatures only when a pool receives many tokens per Δ. With P pools the issuer makes at most P/Δ signatures per second, independent of the token rate. Pooling lets one issuer key mint tokens for every tenant in the pool. It is therefore appropriate only when the tenants already delegate issuance to the CDN or belong to one trust domain. Tenant is part of the claims and of the leaf tweak, so leaves cannot be replayed across tenants.

Verification is given in Algorithm 1. Revocation uses short $V_{\max}$ (1 h by default), signed lists of revoked (e, i) pairs at 8 B each, or withdrawal of a whole epoch root.

C. Root Dissemination

An edge cannot verify an EMBER token until it holds R_e. There are four ways to get it there. **Pull**: the edge fetches SR_e through the shield on a miss, with coalesced fetches. **Push**: the issuer fans each root out to all edges, at fleet bandwidth $BW_{\text{push}} = T_{\text{active}}\, E\,(|\sigma|+40)/\Delta$ for T_{active} active pools. **Period roll-up**: every m epochs one signed bundle lists the m roots. **Stapling**: the client sends SR_e in a header, either only on the session's first request (*staple-first*, our earlier design) or on the first request of every connection (*staple-all*). The simulator models staple-all as stapling on every new edge contact. The stapled header is the 2,461-B signed root (3,298 B as an HTTP/1.1 line). With staple-all, verification survives an outage of the root service or shield tier, during which pull-only edges, which fail closed, reject every token of an uncached epoch; edges depend instead on retaining verified entries, or on the recovery retry. For Poisson token arrivals in pull mode, the expected number of root misses per token at edges with traffic shares w_v is

$$M(\Delta) = \frac{1}{\lambda\Delta} \sum_{v=1}^{E} \big(1 - e^{-\lambda\Delta w_v}\big). \quad (2)$$

The added delay per token is $D = \Delta/2 + t_{\text{epoch}} + M(\Delta) F_{\text{miss}}$, where F_{miss} is the root round trip plus t_v (pull), or t_v alone (stapling). When $\lambda\Delta \ll E$, $M \to 1$: almost every token is the first at its edge in its epoch, so only stapling or push avoids the round trip. When $\lambda\Delta w_v \gg 1$ at every edge, $M \approx E/(\lambda\Delta)$ and pull alone rarely misses.

D. Security

Theorem 1. *Suppose an adversary $\mathcal{A}$ makes q_h queries to* Th *and the issuer evaluates* Th *at $p \leq \sum_e (2N_e - 1)$ tree positions. Then there exist $\mathcal{B}_1, \mathcal{B}_2$ with about the same running time such that* $\text{Adv}^{\text{uf-tok}}_{\text{EMBER}}(\mathcal{A}) \leq \text{Adv}^{\text{euf-cma}}_{\mathsf{Sig}}(\mathcal{B}_1) + \text{Adv}^{\text{sm-tcr}}_{\mathsf{Th},p}(\mathcal{B}_2)$.

Proof sketch: Let R^* be the root the edge used. If the issuer never produced R^* for (kid, e^*), then the verified σ is a forgery, which $\mathcal{B}_1$ outputs. Otherwise $R^* = R_{e^*}$ is honest, and since $i^* < N_{e^*}$ is enforced, an honest leaf i^* exists whose claims differ from c^*. The honest and adversarial hash chains start from different inputs and reach the same root. So at some position they have different inputs and equal outputs under the same tweak. The honest input there contains the salt r_{i^*}, which was chosen after $\mathcal{A}$ fixed c_{i^*}, or a value derived from it. $\mathcal{A}$ therefore hits a fixed honest target, which is an sm-tcr break; distinct tweaks rule out multi-target speed-ups [40]. Replays across epochs, tenants or kid change the tweak and fall into the second case. ∎

This is a sketch, not a new proof. For $n = 32$, plain collision resistance suffices, matching batch signatures [12]. The root and period-bundle signatures use distinct context strings. The hybrid root is an AND-combiner, at least as strong as its stronger component [31]. sm-tcr is defined for a keyed hash family. Our tweak contains kid and the epoch but no random public seed (SLH-DSA's PK.seed). We therefore recommend adding a 16-B random seed per kid to the tweak, carried in SR_e. This adds 16 B to the staple and nothing to tokens.

TABLE I
SCHEMES COMPARED: 13 BASELINES, M2 AND VARIANTS. ALL ARE IMPLEMENTED IN PYTHON; † MARKS SCHEMES THAT ARE ALSO IN THE GO PROTOTYPE.

ID	Scheme
B1†	HMAC-SHA256, key shared with all edges (EdgeAuth-like)
B2†, B3†, B4	Ed25519 (Media CDN); ECDSA P-256 and RSA-2048 (CloudFront)
B5†, B6†, B7, B8	per-token ML-DSA-44/65, SLH-DSA-128s/128f
B9†	per-token hybrid Ed25519 + ML-DSA-44
B10	multi-MAC: one 16-B HMAC tag per verifier domain, $k \in \{1, 3, 10\}$
B11†	delegated Ed25519 key, certified per lifetime L by ML-DSA-44
B12†	per-epoch HMAC key wrapped with ML-KEM-768 [47] to each edge
B13	MTL mode, one series per key [33]
M2†	**stapled per-token ML-DSA-44** (Section IV)
B5v†, B5c	B5 with an edge verify cache; B5 split over two cookies
O1†/O2/O3/O4†	EMBER, $n = 16$/$n = 32$/SLH-DSA-128s root/hybrid root

We did not measure this variant. *Truncation:* ML-DSA-44 is a category-2 parameter set [9]. With $n = 16$, the generic second-preimage bound is about 2^{128} classically and 2^{64} with Grover. The *token* therefore offers category-1 security, the same generic level as SLH-DSA-128's 16-B hashes [21]. Use $n = 32$ for higher categories (+341 B at $N = 2^{16}$). The issuer-chosen salt is mandatory, because $\mathcal{A}$ must not control both inputs of a collision.

Other threats. Downgrade: once a tenant has switched, edges must reject classical tokens for that tenant. Privacy: N_e in a public root leaks issuance volume, and the pair (e, i) identifies a token, as a per-token signature already does. Padding the tree to 2^d hides the volume, at 12.45% issuer slowdown at $N = 1000$ (Python) and up to 38.9% (analytic, $N = 2^{14} + 1$). DoS: tokens naming fake epochs trigger root fetches, and fake staples trigger signature verifications (Section VI-I). Replaying a valid staple costs at most one verification per (kid, e) and edge; per-client limits bound both cases.

V. EVALUATION METHODOLOGY

Schemes. Table I lists the 13 baselines (B1–B13), M2 and the variants. Macaroons and DPoP are discussed only qualitatively (Section II). In B13 the MTL series grows over $V_{\max}$. An MTL series restarted at every epoch is structurally the same as EMBER, so we do not list it separately.

Testbed and implementations. All experiments ran on one KVM guest with 4 vCPUs (Xeon at 2.1 GHz, AVX-512, SHA-NI) and 15 GB of RAM, running Linux 6.18. Primitive costs come from a native C benchmark against OpenSSL 3.5.4 (EVP interface, 5 fresh processes pinned to one core). We report the median of the run medians together with the [min–max] of the run medians. The Go prototype uses Go 1.24.7 and cloudflare/circl 1.6.1, and Python (pqcrypto 1.0.0) is a secondary source. We ran the native suite twice, separated by a VM reboot. Absolute values differ by 15–20% (ML-DSA-44 verification took 100.9 vs. 122.3 µs), while ratios are stable. All tables and all simulations use the second run, except the epoch-controller study and the simulator consistency check (Appendix A). Rerunning the sweep with second-run costs left misses unchanged and moved D by at most 0.1 ms.

Prototype. The Go prototype consists of an edge (`net/http`; root cache with coalescing and a negative cache; optional verified-token cache; V virtual edges), an issuer and root service, and an open-loop load generator. Client, edge and origin run in separate network namespaces joined by userspace link emulators (`netem` is unavailable): one-way delays of 10 ms and 37.5 ms, 1 Gbit/s, MTU 1500. *P1* measures edge CPU cost. It is a closed loop over the edge namespace's loopback with the edge on one core, 64 keep-alive connections, $K = 150$ requests per token and 5 runs. Its primary metric is edge CPU time per HTTP request. P1 measures CPU, not the network. *P3* measures root dissemination over the emulated links with $V \in \{10, 50\}$ virtual edges, $\Delta \in \{0.1, 1, 10\}$ s and $\lambda \in \{20, 200\}$/s, in pull and staple-first modes: 12 configurations, each with a 60 s window and 5 runs. P3 covers EMBER pull and staple-first only. A second prototype campaign (R2) ran the recommended configurations, M2 with its retry path and pooled EMBER (2^5 trees) with staple-all, in P1, in *P2* (open-loop latency at 50% and 80% of each scheme's capacity across the emulated link) and in *P4*. P4 measures recovery: 200 tokens/s per scheme (EMBER with $\Delta = 100$ ms), 10 virtual edges, $p_{\text{move}} = 0.01$, a 30-s window with a cache flush, an edge restart or a 64- or 256-entry cache cap, and 5 runs per scenario. TLS is not included.

Wire bytes and real clients. Request-header bytes per session (1 manifest and 150 segment requests) are encoded with Go's HPACK at 4096-B and 64 KiB tables. To observe real encoders, a raw-frame HTTP/2 server advertises a table size and logs each request's header-block length while a client fetches 150 objects with a 3,311-B cookie value. The clients are headless Chrome 153 (3 runs), curl 8.5.0/nghttp2 1.59.0 and the Go x/net client. We did not test Firefox or Safari.

Fleet simulator. The simulator is an epoch-level Monte Carlo engine driven by the native costs. Its default fleet has $E = 500$ edges with Zipf(0.8) traffic shares and 50 shields, and $T = 1000$ tenants with Zipf(1.0) rates and aggregate rate $\bar{\lambda} \in \{200, 2000, 20{,}000\}$/s. Sessions have K geometric with mean 150, 4 s apart, and each segment is re-routed to another edge with probability $p_{\text{move}} = 0.0005$ (about 7% of sessions move once). RTTs are lognormal: edge–shield 15 ms, shield–root 60 ms, token receipt to first request 120 ms. All of these are assumptions; the sensitive ones are swept (Appendix A). The pooling policies are per-tenant trees, 8 shards, 10 or 50 regional pools, and one pool per issuer key. Each is run with a fixed Δ, work-conserving closing (WC, close whenever the signer is idle) or M2. CIs are Student-t 95% intervals over 5 seeds with common random numbers. Unless stated, their half-widths are below 1% of the mean.

Trace replay. We replay Meta's public CacheLib CDN traces (`cdn/202303`, Apache-2.0) [14], [15]: three edge clusters (eag, nha, prn), the first three full days of each (15–18 March 2023), giving 9 cluster-day replicates with Student-t 95% CIs. The trace has timestamps, anonymized object keys,

TABLE II
NATIVE PRIMITIVE COSTS (OPENSSL 3.5.4, SECOND RUN): MEDIAN OF 5 RUN MEDIANS [MIN–MAX OF RUN MEDIANS], µS. KEY SIZES ARE RAW KEYS (ECDSA: 65-B POINT; RSA: 270-B MODULUS AND EXPONENT), NOT SPKI.

Scheme	pk	sig	sign	verify
HMAC (200 B)	–	32	0.30 [0.30–0.30]	0.30 [0.30–0.30]
Ed25519	32	64	30.1 [30.1–30.4]	94.6 [93.8–96.2]
ECDSA P-256	65	71	23.0 [22.9–31.1]	69.6 [69.5–70.0]
ML-DSA-44	1312	2420	408.8 [386.9–452.4]	100.9 [100.2–120.4]
ML-DSA-65	1952	3309	676.5 [673.5–682.7]	154.8 [154.6–165.8]
SLH-DSA-128s	32	7856	306,464	288.1 [282.4–320.0]
Path d=0 / d=16	–	–	–	0.24 / 2.34

content-type and VIP-type ids and a sampling rate (10^{-4}; $4 \cdot 10^{-4}$ for nha), but no client, session or host identifier. We must therefore *assume* the token granularity, and this assumption decides the edge result: **one signed-URL token per object key per hour**, issued at the key's first request in each hour, so K is the key's requests in that hour. Content types (23–24) stand in for tenant pools, VIP types (10–12) for trust domains, and one pool per cluster for a CDN-wide key. *Sampled* results replay the published trace exactly. *Scaled* results multiply each pool's per-minute token count by the inverse sampling rate with Poisson arrivals within the minute, which assumes key-hash sampling; at the sampled rate this model reproduces the exact replay within 0.7% in signatures/s and 0.6% in edge CPU. With no host field, we bound the edge with three verifier models: one cluster-wide shared cache, tokens hashed to one of 1000 hosts, and requests spread over 1000 hosts. The documented traffic factors [15] imply about 390–2,200 hosts per cluster.

Hypotheses. We fixed hypotheses H1–H9 in the design document before running any experiment (repository commit `3148c65`, 2026-11-05 18:07 UTC). Appendix B (Table IX) reports every outcome.

VI. RESULTS

A. Primitive Costs and Naive Post-Quantum Tokens

Table II lists the native costs. ML-DSA-44 verification takes 100.9 µs, about the same as Ed25519 (94.6 µs). An EMBER path of depth $d = 16$ takes 2.34 µs and $d = 0$ takes 0.24 µs. HMAC over 200 B takes 0.30 µs. A path at $d = 16$ is thus 43× cheaper than OpenSSL's ML-DSA-44 verification but 7.9× HMAC. In Go, a whole EMBER token verification takes 3.07 µs, against 32.76 µs for circl's ML-DSA-44, a ratio of 10.7×. circl verifies 3.6–4.0× faster than OpenSSL's portable code.

The token sizes below are measured with a 60-character path. Per-token signatures are large: 3,350 B for B5, 4,535 B for B6 and 10,598 B for B7. B5 fits RFC 9110's 8000-octet URL and RFC 6265's 4096-B cookie but not a 2 KB legacy limit. B6 exceeds the cookie minimum. The classical and symmetric baselines are small: 166 B (B1), 209 B (B2), 214 B (B11) and 171 B (B12). Splitting B5 over two cookies (B5c) gives at most 1,687 B per cookie.

TABLE III
WHOLE-REQUEST EDGE COST IN THE GO PROTOTYPE (P1; ONE EDGE CORE, SAME SESSIONS FOR ALL SCHEMES, $K = 150$; MEAN [95% t-CI] OVER 5 RUNS) IN TWO CAMPAIGNS. R2: THE RECOMMENDED CONFIGURATIONS, WITH M2 USING THE STAPLE-REQUIRED RETRY PATH AND POOLED EMBER A 2^5 TREE. THE PROTOTYPE USES A SHORTER CLAIM STRING THAN THE ENCODER TABLE (B1 100 B VS. 166 B; M2 83 B VS. 148 B).

	token	edge CPU (µs/req)	
Scheme	(B)	R1	R2
B1 HMAC	100	22.80 [22.23, 23.36]	25.90 [25.03, 26.76]
M2 stapled	83	**22.33** [21.60, 23.05]	**25.35** [24.01, 26.70]
O1 EMBER 2^{10}, warm	309	26.16 [25.51, 26.82]	–
O1 EMBER pooled 2^5	202	–	27.23 [26.50, 27.96]
B5v + cache	3,285	42.53 [41.89, 43.18]	–
B5 ML-DSA-44	3,284	75.58 [72.12, 79.04]	77.52 [73.79, 81.25]

B. Stapling at the Edge with Session Tokens

Table III compares whole HTTP requests, including Go's request parsing, on one edge core, for session tokens that carry $K = 150$ requests each. With stapling, per-token ML-DSA-44 (M2) costs 22.33 µs of edge CPU per request, which is indistinguishable from HMAC (22.80 µs). Part of this parity comes from M2's 16-B nonce reference being shorter than the 32-B HMAC tag (a truncated tag would shrink HMAC tokens too). In one run the edge recorded 7,419 staple verifications in 1,108,422 requests. EMBER with a warm 2^{10} tree costs 26.16 µs, 17% more than M2, because it hashes a path on every request. The R2 campaign, on an otherwise idle VM, ran the recommended configurations. M2 again matched HMAC (25.35 vs. 25.90 µs; throughput ratio 1.02 [0.94, 1.10]), and pooled EMBER with a 2^5 tree cost 27.23 µs, 5% more than HMAC. Across the emulated 20 ms link, B1, M2 and EMBER were link-bound at 24.4–24.7k requests/s, while B5 was edge-bound at 12,226/s. Under open-loop load at 50% and 80% of each scheme's capacity, M2 and pooled EMBER added between -0.11 and $+0.01$ ms to the median latency relative to HMAC, with every CI including zero, while B5 added 0.81 [0.77, 0.84] and 1.55 [0.94, 2.16] ms. B5v still parses and hashes its 3.3 KB token on every request and costs 42.53 µs, and plain B5 costs 75.58 µs. An earlier 12-scheme campaign (Appendix A) puts per-token Ed25519, ECDSA and hybrid tokens above per-token ML-DSA-44 in Go.

Stapling also removes most of the byte cost. M2's URL token is 148 B in the encoder (claims plus a 16-B nonce reference), against 3,350 B for B5. M2 needs no batching wait, so its mean added delay with staple-all is 0.41 ms in simulation (Table IV). For EMBER the edge comparison is unfavorable. M2 pays one 100.9 µs verification per (token, edge), amortized over K requests. EMBER pays one root verification per (root, edge) plus, on every request, $\mathrm{path}(d) - \mathrm{path}(0)$. Once the root is cached, EMBER is cheaper at the edge only if $K < K^*_{\mathrm{cpu}} = 100.9/(\mathrm{path}(d) - \mathrm{path}(0))$, which gives $K^*_{\mathrm{cpu}} = 154$ for $d = 5$ and 96 for $d = 8$. Our synthetic workload has a mean K of 150, close to the crossover for $d = 5$ and above it for deeper trees. The simulator agrees: per-request edge crypto is 0.953 µs for M2 against 1.509 µs for pooled EMBER at $\bar{\lambda} = 2000$/s. Both are microsecond-level.

TABLE IV
OPERATING POINTS WITH EVERY METRIC AT THE SAME CONFIGURATION (SIMULATED; STAPLE-ALL; TOKEN SIZE FROM THE ENCODER). ISSUER: CORES OF ML-DSA-44 SIGNING AND TREE WORK. EDGE: CRYPTO µS PER REQUEST. HOT: ML-DSA VERIFICATIONS/S AT THE BUSIEST EDGE. D: MEAN ADDED DELAY.

Configuration	N_e=1 (%)	tok/tree (mean)	token (B)	issuer (cores)	edge (µs)	hot (/s)	D (ms)
$\bar{\lambda} = 2000$ tokens/s							
M2 per-token	100	1	148	0.818	0.95	165.8	0.41
per-tenant, 10 ms	71.5	1.22	161	0.674	1.01	162.4	5.41
per-tenant, WC	99.8	1.00	161	0.818	0.95	165.8	0.41
8 shards, 10 ms	8.8	2.79	204	0.295	1.20	150.1	5.41
1 pool, WC	63.6	1.26	161	0.650	1.01	162.6	0.54
1 pool, 10 ms	0	20.0	268	**0.0425**	1.51	88.1	5.43
1 pool, 100 ms	0	200	332	0.0057	1.71	15.2	50.6
$\bar{\lambda} = 20{,}000$ tokens/s							
M2 per-token	100	1	148	8.18	0.95	1,658	0.41
per-tenant, 10 ms	41.0	1.85	183	4.43	1.17	1,420	5.41
1 pool, WC	0.03	8.31	247	1.000	1.38	1,244	0.62
1 pool, 10 ms	0	200	332	**0.057**	1.71	151	5.57
$\bar{\lambda} = 200$ tokens/s							
M2 per-token	100	1	148	0.082	0.95	16.6	0.41
1 pool, 10 ms	13.6	2.31	204	0.036	1.16	15.4	5.41

This edge result holds only for long-lived tokens; Section VI-D shows that it reverses when tokens carry few requests.

C. Batching Relieves the Issuer, Only When the Pool Is Busy

Fig. 2 and Table IV answer where batching pays. Take the per-tenant, 10 ms configuration that our earlier version recommended, at $\bar{\lambda} = 2000$/s. There, 71.5% of tokens sit in single-token trees and trees hold 1.22 tokens on average. Even in the busiest tenant decile, which carries 69% of the tokens, 59.4% of tokens are alone in their tree. The issuer makes 1,646 root signatures/s for 2,000 tokens/s: 0.674 cores against 0.818 for per-token signing, a 1.2× saving, and work-conserving closing is worse. Pooling all tenants of an issuer key into one tree per 10 ms changes this: each tree holds 20 tokens on average (200 at 20,000/s), and the issuer makes at most $1/\Delta = 100$ signatures/s regardless of the rate. Issuer CPU falls to 0.0425 cores at $\bar{\lambda} = 2000$/s, 19× less than M2, and to 0.057 cores at 20,000/s, 144× less. Root verifications at the busiest edge fall from 1,658/s to 151/s. The costs are a larger URL token (268 or 332 B instead of 148 B), $\Delta/2$ of issuance wait (5.4 ms mean), and 1.6–1.8× the fleet edge crypto CPU (0.456 vs. 0.288 cores at $\bar{\lambda} = 2000$/s; 5.18 vs. 2.88 at 20,000/s). Regional pools reduce verifications at the hottest edges (89.5/s with 50 regions) but help the issuer little, because each region's rate is small. At $\bar{\lambda} = 200$/s, pooling saves 0.046 cores, which does not justify the extra delay and bytes. Issuer cores use OpenSSL's portable ML-DSA-44 signing (408.8 µs). circl signs in 121.2 µs (root) and 123.9 µs (token), so absolute issuer cores fall about 3.4-fold and the rate at which per-token signing needs 0.41 cores moves from about 10^3 to 3.4×10^3 tokens/s per key; the pooled-to-M2 ratios change much less.

Microbenchmark capability. A native issuer core produces 41,774 tokens/s at $N = 16$ (bootstrap 95% CI 18,248–50,049), 469,254 at $N = 2^8$ and 1,275,990 at $N = 2^{14}$ (460-B tokens), against 2,446 for per-token ML-DSA-44 signing. $N \geq 2^{14}$ requires $\lambda\Delta \geq 1.6 \times 10^4$, which no simulated configuration with $\Delta \leq 0.1$ s attains.

TABLE V
REPLAY OF META'S CDN TRACES (3 CLUSTERS × 3 DAYS; MEAN [95% t-CI] OVER THE 9 CLUSTER-DAYS). **ASSUMPTION: ONE SIGNED-URL TOKEN PER OBJECT PER HOUR.** $\Delta = 10$ MS, RATES SCALED TO THE FULL CLUSTER (SAMPLED REPLAY IN THE LAST TWO ROWS). EDGE: CRYPTO µS PER REQUEST WITH A SHARED VERIFIER / WITH EACH REQUEST AT A RANDOM ONE OF 1000 HOSTS. BYTES: URL TOKEN PLUS STAPLES, PER REQUEST, SHARED VERIFIER.

Scheme / pool	N_e=1 (%)	med. N_e	token (B)	issuer (cores)	edge (µs/req)	bytes (B/req)
M2 per-token	100	1	148	176.7	53.8 / 87.3	1,869
EMBER per type	0.02	547	335	1.08	1.6 / 61.8	345
EMBER per VIP	0.02	2,164	390	0.73	1.8 / 38.7	394
EMBER cluster	0	4,369	425	**0.38**	**1.9** / 17.6	**426**
sampled trace as published (66 tokens/s)						
EMBER per type	91.5	1	163	0.0256	51.4 / 87.3	1,833
M2 per-token	100	1	148	0.0269	53.8 / 87.3	1,869

D. Real CDN Trace: Requests per Token Decide the Edge

The synthetic workload assumes long sessions ($K = 150$) and 200–20,000 tokens/s CDN-wide. Table V replays real traffic under the assumption of Section V, one signed-URL token per object per hour. Then a token carries only 1.7–2.4 requests on average (2.36 [2.14, 2.57] for eag, 1.71 [1.47, 1.95] for nha, 1.72 [1.69, 1.76] for prn), 78–84% of tokens are used once, and the request-weighted median K is 2–4. Scaled to the full cluster, the token rate is 2.3×10^5–7.7×10^5/s per cluster. That is 100–400× the whole synthetic CDN at $\bar{\lambda} = 2000$/s.

Issuer. At these rates batching works even without cross-tenant pooling. With one tree per content type per 10 ms, only 0.02% of tokens are alone in their tree and the median tree holds 547 tokens. The issuer needs 1.08 cores [0.91, 1.26] against 176.7 [96.9, 256.6] for per-token signing, and 0.38 cores [0.23, 0.54] with one pool per cluster. With one pool per cluster, per-token leaf and encoding work (0.76 µs per token, about 0.33 cores) dominates, not signatures. On the trace as published (66 tokens/s, a CDN thinned by the sampling rate), 91.5% of per-type tokens sit in single-token trees and EMBER ties M2 (0.0256 vs. 0.0269 cores), which reproduces the degeneracy of Section VI-C. The pool's token rate, not the pooling structure, decides: from the measured costs, EMBER's issuer is at least 10× cheaper than per-token signing above 1,021 tokens/s per pool at $\Delta = 10$ ms, and 99.5–99.9% of trace tokens are in content-type pools above 10^3/s. Trees of $N \geq 2^{14}$ appear only at $\Delta = 1$ s (median 54,780 per type; 477-B tokens). With $V = 10$ min or 24 h (nha), per-type trees still hold medians of 312 and 245 tokens.

Edge. With $K \approx 2$, M2 pays one ML-DSA-44 verification about every second request: 53.8 µs [47.4, 60.1] of edge crypto per request with a shared verifier, against 1.88 µs [1.80, 1.96] for the cluster-pooled EMBER, whose root a verifier meets once per epoch rather than once per token. M2 also sends a 3,242-B staple with almost every token, so it averages 1,869 B per request against 426 B. EMBER stays cheaper when verification is spread over 1000 hosts (16.7–17.6 vs.

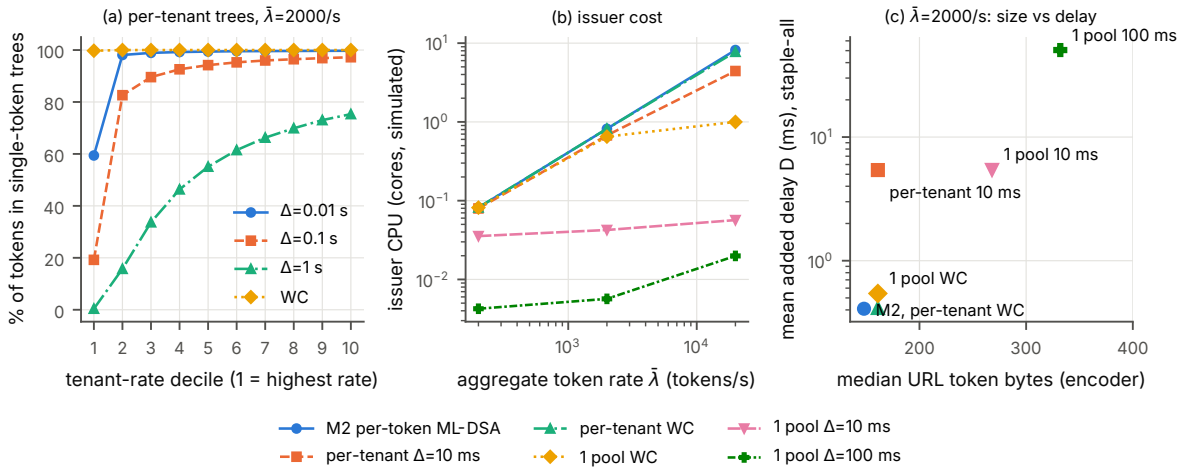


Fig. 2. Operating region (simulated, second-run costs, 5 seeds). (a) Share of tokens in single-token trees by tenant-rate decile, per-tenant trees. (b) Issuer CPU against aggregate rate. (c) Median URL token size (encoder) against mean added delay with staple-all.

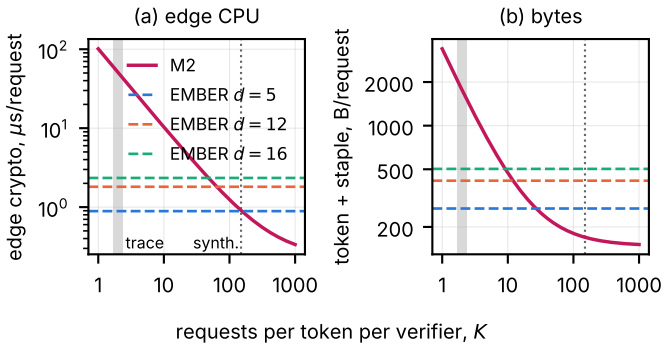


Fig. 3. Edge cost per request against requests per token per verifier (K), analytic from the native costs, with the root already held by the verifier. M2: one ML-DSA-44 verification and one 3,242-B staple per token. EMBER: one path per request. Gray band: mean K on the trace (1.71–2.36). Dotted line: synthetic mean $K = 150$.

TABLE VI
REQUEST-HEADER BYTES PER SESSION (1 + 150 REQUESTS ON ONE CONNECTION; GO HPACK ENCODER AT 4096-B AND 64 KIB TABLES; SAME PLACEMENT FOR ALL SCHEMES). EACH REQUEST ALSO CARRIES ABOUT 570 B OF OTHER, CHROME-LIKE HEADERS, INCLUDING A CHANGING :PATH. THESE EVICT A 3.3 KB COOKIE FROM A 4096-B TABLE.

	URL, HTTP/2		Cookie, HTTP/2	
Scheme	4096 B	64 KiB	4096 B	64 KiB
B1 HMAC	26,706	24,918	9,148	9,121
B11 deleg. Ed25519	33,277	30,825	9,308	9,161
B5 ML-DSA-44	457,831	410,359	458,568	11,667
B5c (2 cookies)	–	–	460,833	11,932
M2 + staple	27,048	25,273	12,114	11,711
EMBER 2^5 + staple	43,401	39,966	13,165	11,853
EMBER 2^8 + staple	52,389	47,958	13,466	11,894
EMBER 2^{14} + staple	69,161	63,012	14,090	12,008

53.8–87.3 µs), and in every verifier model at the scaled rates. Verifier contacts, the requests that must carry a staple or would miss in pull mode, are 0.53–0.86 per request for M2 and 0.0002–0.16 for the cluster pool. These byte figures are lower bounds that assume a client staples exactly at verifier contacts.

The crossover. Fig. 3 generalizes both results with the measured costs. Once a verifier holds the root, M2 costs $t_v/K + \mathrm{path}(0)$ per request and EMBER costs $\mathrm{path}(d)$. M2 is therefore cheaper in edge CPU only when $K > K^*_{\mathrm{cpu}} = t_v/(\mathrm{path}(d) - \mathrm{path}(0))$: 154, 64 and 48 requests for $d = 5$, 12 and 16. On bytes, M2 sends $148 + 3242/K$ and EMBER $S_{\mathrm{tok}}(d)$ (Eq. (1)), so M2 is smaller only above $K^*_{\mathrm{bytes}} = 3242/(S_{\mathrm{tok}}(d) - 148)$: 27, 12 and 9 requests. The synthetic sessions ($K = 150$) sit near or above both crossovers, and the trace's per-object tokens ($K \approx 2$) far below them. Which design is cheaper at the edge is therefore a property of the token granularity, which the operator chooses and the trace cannot reveal.

E. Bytes on the Wire and Real HPACK Encoders

Table VI compares schemes in the same placement. In URL form, stapled EMBER at the pooled tree size 2^5 uses 0.095× B5's bytes under HTTP/2 with a 4096-B table, but M2 uses less still: 27,048 B, within 1.3% of HMAC. In cookie form, the HPACK table size decides the outcome. With a 64 KiB table, even unstapled B5 (11,667 B) is smaller than stapled EMBER (11,853 B). Every post-quantum option then costs 11.7–12.0 KB per session, close to HMAC's 9.1 KB. The stapled header costs EMBER 2,968 B per HTTP/2 session (3,298 B as an HTTP/1.1 header line) and M2 2,921 B.

Which table size applies depends on the client's encoder (Section II). When our server advertised 64 KiB, Chrome 153 adopted it in all 3 runs, and the 3.3 KB cookie then cost 25.7 B per request after the first (about 6.5 KB for 150 requests). With a 4096-B table Chrome re-sent it at 2,993–3,000 B per request. curl/nghttp2 and the Go client clamped the table to 4096 B whatever the server advertised (the Go source has a 4096-B default encoder limit). Even so, Go kept the cookie indexed at 228 B per request, because its own header set is small, while

TABLE VII
DISSEMINATION MODES, EMBER AT $\bar{\lambda} = 2000$/s, $\Delta = 10$ MS (SIMULATED, 5 SEEDS). T: PER-TENANT TREES. P8: 8 POOLED SHARDS. WAIT: SHARE OF REQUESTS THAT WAIT FOR A ROOT.

	Per-tenant trees (T)				8 shards (P8)	
Mode	miss/tok	D (ms)	wait %	root/served	miss/tok	D (ms)
pull	1.069	96.1	0.710	$2.4 \cdot 10^{-5}$	1.054	94.3
staple-first	0.073	11.6	0.049	$1.6 \cdot 10^{-6}$	0.071	11.5
push	0.622	19.7	0.412	$5.0 \cdot 10^{-3}$	0.616	19.6
pull+roll-up	1.045	93.2	0.694	$4.5 \cdot 10^{-5}$	1.003	89.8
push+roll-up	1.012	91.3	0.672	$1.0 \cdot 10^{-4}$	0.997	89.7

curl re-sent it (2,671 B per request). Whether a large cookie stays indexed thus depends on the encoder's table limit, its indexing policy and the other headers; a CDN that advertises a large table cannot rely on it. Stapled designs do not depend on this setting.

F. Dissemination and the Stapling Assumption

Table VII compares the dissemination modes with per-tenant and pooled trees side by side. Pull makes nearly every first request wait for a root (1.069 misses per token), because $\lambda\Delta \ll E$ (Eq. (2)). Per-tenant push costs $5.0 \cdot 10^{-3}$ of served bytes and 2.27 GB/s of fleet root traffic, and it still misses 0.622 times per token, because the 150 ms push delay exceeds the 120 ms between token receipt and first request. Roll-up removes only 1.4–2.3% of misses: from 6,098 to 6,011 per 10^6 requests at $\Delta = 1$ s, and from 1.069 to 1.045 per token at 10 ms. Roots arrive before any period closes, so roll-up is useful only for carrying revocation lists and bounding root state. Root traffic is at most $1.0 \cdot 10^{-4}$ of served bytes in every mode except per-tenant push.

The staple-first figure of 0.049% waiting requests is a *model output that rests on one assumption*. Its misses come entirely from the assumed mid-session re-route probability p_{move} (Fig. 5). The share is 0 at $p_{\text{move}} = 0$, 0.049% at the default $5 \cdot 10^{-4}$, 0.48% at $5 \cdot 10^{-3}$ and 0.95% at 10^{-2}, and M2 behaves the same. *Staple-all* removes waiting entirely at every p_{move}, at a cost of 2,501 to 6,254 B per session of raw signed-root bytes (simulator; about 3.3 KB per staple as an HTTP/1.1 header line). We recommend staple-all. Our prototype did not exercise re-routes.

G. When Delegated Ed25519 (B11) Suffices

B11 is the strongest classical competitor. Its token is 214 B, its issuer signs in 30.1 µs per token, and its certificate traffic is 0.383 MB/s at $L = 1$ h, against 8.41 MB/s for EMBER pull. A delegated Ed25519 key is exposed from the publication of its certificate until the last token signed under it expires, a window of $L + V_{\max}$, so B11 is quantum-safe only if breaking one key takes longer than $L + V_{\max}$ (Fig. 6). Shortening L is cheap: at $L = 60$ s the cost is 16.7 certificate fetches per second per edge, 0.17% of an edge core and 23 MB/s of fleet traffic. The binding constraint is $V_{\max}$. One published estimate for a 256-bit elliptic-curve key gives one key every 38 h on a strictly 2D-local architecture, and 300–700× less with non-local connections, about one key every 10 minutes in one regime [48]. These concern hypothetical machines and say nothing about when they will exist, but the second figure is below our $V_{\max} = 1$ h. Operators who use B11 should therefore shorten $V_{\max}$, for example to 10 min, rather than rely on L. On issuer cost, B11 (30.1 µs/token) beats M2 (408.8) and per-tenant EMBER (337.2), but loses to pooled EMBER (21.2 at 2,000/s, 2.83 at 20,000/s).

H. Simulator Validation Against the Prototype

We ran the simulator with the prototype's P3 configuration and measured RTT distribution (Fig. 4). P3 has 12 configurations in each of two modes, 24 rows per metric (misses per token, root bytes per token). P3 counts a stapled-root install as a miss, so both modes give the same miss counts and there are 12 independent comparisons per metric, with the same miss definition on both sides. All 24 rows pass for both metrics (relative error at most 15% or the simulator inside the prototype's CI), with a maximum error of 4.4%. Mean miss latency passes for pull at $\Delta \leq 1$ s (76.9–78.5 ms measured against 75.9 ms simulated) and fails elsewhere: at $\Delta = 10$ s the prototype measures 118–137 ms, because timer-aligned epochs release about 4,400 tokens whose first requests queue at the one-core edge, which the simulator does not model. Stapled installs take 0.074–0.086 ms against a modelled 0.026 ms, because the model omits decoding the 3.3 KB header. The validation covers neither re-routes nor the shield tier, so we claim it only for miss and byte counts at small scale ($V \leq 50$).

I. Other Results

DoS (one simulated edge, 10^4 fake tokens/s). Fake epochs spread over all tenants cause 9,270–9,636 root fetches/s, 4,630 verifications/s and 0.484 of a core; coalescing and negative caching do not help, because keys rarely repeat, although for one low-rate tenant coalescing cuts fetches from 7,680/s to 65/s. A flood of fake staples costs 1.026 cores, and a limit of one staple per second per client (100 clients) reduces this to 0.0269 cores, so per-client limits are required for staples and fetches. *Hybrid root.* At $\Delta = 1$ s in pull mode, verifying the Ed25519 half of a hybrid root raises simulated edge CPU from 1.405 to 1.982 µs per request (+41.1%), because a root is verified about once per 141 requests; in the warm P1 configuration O4 and O1 differ by 0.09%. *End-to-end.* The mean added latency per request is 0.0503 ms for EMBER staple-first, 0.0049 ms for B5v, 0.101 ms for B5 and 0.486 ms for EMBER pull (p99.9 112 ms); B5v beats stapled EMBER by 10× on the mean, and all schemes are below 1 ms at p99.

VII. DISCUSSION

A. Deployment Guidance

Table VIII summarizes the trust and cost trade-offs. Our recommendations follow from the measurements.

1. If edges may hold a minting key, use HMAC. HMAC (B1) is post-quantum and the cheapest option. Per-domain multi-MAC (B10) confines a multi-CDN breach to one vendor at HMAC cost.

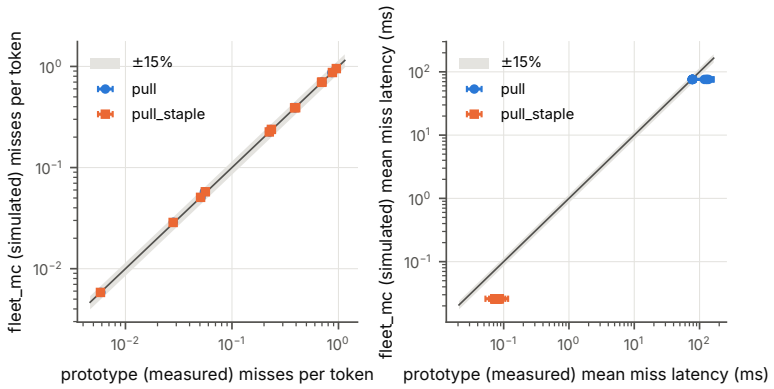


Fig. 4. Simulator (50 seeds, fed the measured RTTs) against the prototype (P3, 5 runs, 95% CI) for 12 configurations in pull and staple-first modes. Left: misses per token. Right: mean miss latency.

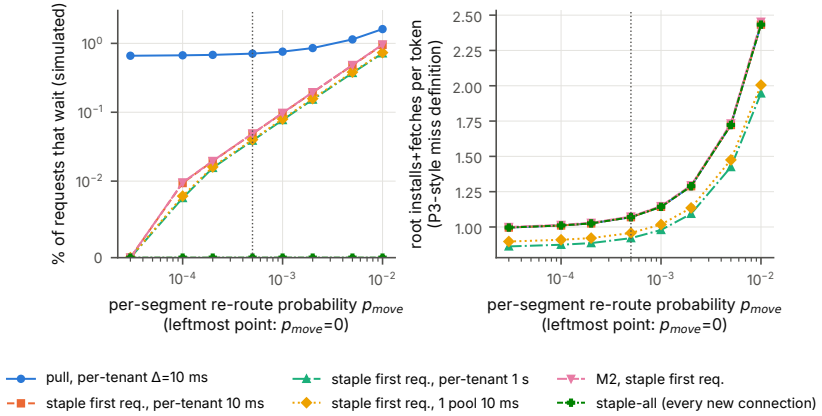


Fig. 5. Sensitivity of stapling to mid-session re-routes (simulated, $\bar{\lambda} = 2000$/s). Left: share of requests that wait for a root or signature. Right: root installs plus fetches per token (the prototype's miss definition). For pull this is the fetch count (1.0–2.4 per token), and it coincides with the staple-all curve. M2 and EMBER staple-first overlap almost exactly in both panels. The dotted line marks the default $p_{\text{move}} = 0.0005$.

2. Otherwise, first measure K, the requests a token carries at one verifier. Above $K^*_{\text{cpu}} \approx 48$–$154$ and $K^*_{\text{bytes}} \approx 9$–$27$ (tree depth 16 to 5), stapled M2 is cheaper at the edge; below them, a pooled EMBER tree is (Fig. 3). Session-scoped streaming tokens are in the first regime; one token per object, as assumed for the trace, is in the second, where M2 costs 28× more edge crypto and 4.4× more bytes (Table V).

3. With long-lived tokens (large K), staple. M2 matches HMAC in edge CPU (Table III) and is the smallest post-quantum option in URL form, and within 0.4% of it in cookie form (Table VI). Clients should staple on every new connection (Fig. 5). Edges must size their verified-token cache for the live token set (at most 14.3 MB of payload at the busiest simulated edge for $\bar{\lambda} = 2000$/s): a recovery adds about 36 ms (1.8 round trips) on a live connection and a 258–263 ms reconnect after a restart, and with an undersized cache M2 retried on 55% of requests against 10% for pooled EMBER (Section IV). Above about 10^3 tokens/s per issuer key with OpenSSL's signer, or 3.4×10^3/s with circl's, per-token signing costs more than 0.41 issuer cores, and pooled EMBER with staple-all saves 19–144× of issuer CPU for 1.6–1.8× the fleet edge crypto, 120–184 more bytes per token and 5.4 ms of issuance delay.

4. With short-lived or per-object tokens (small K), pool. Pooled EMBER is then cheaper at both the issuer and the edge, provided the pool is busy: at $\Delta = 10$ ms it makes at most $1/\Delta$ root signatures per second and is at least 10× cheaper than per-token signing above about 10^3 tokens/s per pool. On the trace that held for 99.5–99.9% of tokens even with per-content-type pools, and one pool per cluster needed 0.38 cores where M2 needed 177. Pooling requires a shared issuer key, as with a CDN-operated issuer or a single large content owner. In thin pools both designs pay about one ML-DSA verification per token (51.4 vs. 53.8 µs per request on the sampled trace), and the choice rests on the trust trade-offs of rules 1 and 5.

5. Consider B11 if its assumption holds. Delegated Ed25519 is smaller and cheaper to disseminate, but appropriate only if breaking one key takes longer than $L + V_{\max}$ (Section VI-G). Published estimates span 38 h to about 10 min per key [48], so pair B11 with a short $V_{\max}$ (for example 10 min).

6. Operational settings. Use a node size of $n = 16$ for category-1 tokens and $n = 32$ above that. Add a per-*kid* public

TABLE VIII
TRUST AND COST COMPARISON. BYTES: ENCODER URL TOKEN, 60-CHARACTER PATH. VERIFY: NATIVE OPENSSL 3.5.4 MEDIANS. DISSEMINATION: SIMULATED FLEET TRAFFIC FOR $E = 500$ ($\bar{\lambda} = 2000$/S) OR PER-SESSION STAPLE BYTES.

Scheme	Secret at edge	One compromised edge mints for	PQ assumption	Bytes	Verify (µs)	Dissemination
B1 HMAC	yes (key lifetime)	all edges, all paths of the key	unconditional	166	0.30	none
B10 multi-MAC, k=3	yes	its own domain	unconditional	187	0.30	none
B12 KEM-wrapped HMAC	yes (epoch keys)	all edges, live epochs	unconditional	171	0.30	283.5 MB/s key push
B2 Ed25519	no	none	none (Shor)	209	94.6	none
B11 delegated Ed25519	no	none	no break within $L+V_{\max}$	214	94.6 per (key, edge)	0.383 MB/s ($L = 1$ h)
B5 ML-DSA-44	no	none	unconditional	3,350	100.9 per request	none
B5v + verify cache	no	none	unconditional	3,350	100.9 first, 2.19 hit	none
M2 stapled ML-DSA-44	no	none	unconditional	148	100.9 per (token, edge), $\leq$0.21 hit	staple 2,921 B/session (h2)
EMBER pooled, staple-all	no	none	unconditional	268	0.89 path (d=5) + 100.9 per (root, edge)	staple 2,968 B/session (h2)
EMBER per-tenant, pull	no	none	unconditional	161–247	0.24–0.75 path (d=0–4) + root	8.41 MB/s ($\Delta = 1$ s)

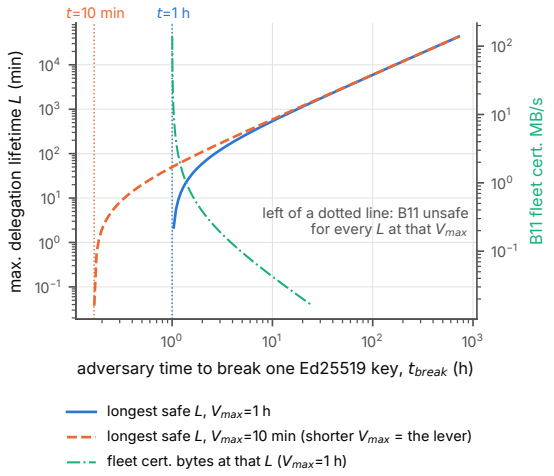


Fig. 6. B11 safety against quantum key recovery (analytic). A delegated key is exposed for its lifetime L plus $V_{\max}$. It is safe only if one Ed25519 key takes longer than $L + V_{\max}$ to break. Right axis: fleet certificate traffic at that L ($E = 500$, $T = 1000$).

seed to the tweak. Set $V_{\max} = 1$ h and a skew tolerance $\tau = 5$ s. Apply per-client limits on staples and fetches, and fetch roots only for configured *kid*s and recent epochs. Do not deploy per-tenant push or rely on roll-up to reduce misses.

B. Limitations

- *One VM.* All prototype components share one 4-vCPU KVM guest, with network namespaces and a userspace link emulator instead of `netem`, no physical NIC and no TLS. P1 measures edge CPU over loopback. Absolute P1 values differed by up to 26% across campaigns (B1: 20.54, 22.80 and 25.90 µs), while M2's ratio to HMAC stayed at 0.98.
- *Two native runs.* Absolute costs differ by 15–20% across a VM reboot; all but the controller study and the consistency check use the second run.
- *Simulator scope.* The simulator is validated only for miss and byte counts, at small scale ($V \leq 50$), without reroutes, a shield tier or queueing, so it under-predicts pull-miss latency for timer-aligned bursts. Its workload parameters are assumptions, and p_{move} alone determines staple-first waiting.
- *Prototype coverage.* The R2 campaign ran M2 and pooled EMBER in P1, P2 and P4, but only at synthetic K (150 in P1, about 20 in P4) and without a shield tier. Issuer cores and dissemination for these designs remain simulated, and production flush and restart rates are unknown.
- *Trace mapping.* One operator, three clusters, three days (March 2023). With no session, client or host identifier, the edge results rest on the assumed token granularity, and the scaled results on key-hash sampling and Poisson arrivals within each minute. Content types stand in for tenants, and re-routes cannot be observed. Under this mapping, the synthetic token rate is 100–400× below one real cluster's.
- *Clients and security.* Firefox and Safari were not tested. We give a proof sketch, not a machine-checked proof; the public-seed variant was not implemented; bearer-token replay is unchanged; pooling concentrates minting in one key.

VIII. CONCLUSION

Post-quantum CDN tokens have two cost centers that respond to different variables. At the issuer, the variable is the token rate per key. Pooled epoch-Merkle batching (EMBER, a CDN profile of MTL-mode batch signing) cuts issuer CPU 19–144× in simulation and needs 0.38–1.08 cores where per-token signing needs 177 at the full token rates implied by Meta's CDN traces. It pays above roughly 10^3 tokens/s per shared key; in thin pools it saves nothing. At the edge, the variable is the number of requests K a token carries at one verifier. With session tokens ($K \approx 150$), stapling the signature once per connection lets per-token ML-DSA-44 cost the edge as much as HMAC, with a 148-B URL token. With one token per object per hour ($K \approx 2$ on the trace), the same design verifies a signature about every second request, and a pooled tree is 28× cheaper in edge crypto and 4.4× smaller on the wire. The crossover lies at $K \approx$ 48–154 for CPU and 9–27 for bytes. Real token granularity is the operator's choice and is not visible in public traces. Stapled designs also make

edges depend on their verified-entry caches: on our prototype, a lost entry costs about 36 ms (1.8 round trips) on a live connection, a restart costs a 258–263 ms reconnect, and M2, which needs one entry per token, is far more sensitive to cache size than a pooled tree. Whether HPACK hides large cookies depends on the client, and delegated Ed25519 keys suffice only with a short $V_{\max}$. Open questions are traces with session or token identifiers, measurements in a multi-PoP deployment with re-routes, a simulator with queueing, and Firefox and Safari encoder behavior.

Data Availability

All code, raw measurements, aggregated CSV files and figure scripts are in the author's repository. `NUMBERS_TRACE.md` maps every number in this paper to its source file and row. [Editable: insert public archive DOI or URL before submission.]

Declaration on the Use of AI Tools

[Editable: AI-based tools were used to assist with code, experiments and drafting. The author reviewed and takes responsibility for all content. Adapt to the venue's policy.]

Acknowledgment

[Editable.]

Appendix A
Secondary Results

Epoch controller (first-run costs). An adaptive controller estimated the token rate (EWMA) and, at each epoch close, chose Δ from a 25-point logarithmic grid in $[0.01, 60]$ s to minimize D under size, signer-capacity and 2 s issuance-latency constraints. A demotion rule fixed in advance applied if it beat the best fixed Δ (chosen in hindsight) by less than 10 ms or 10%. Over a 24-h diurnal workload (pull, per-tenant trees) it chose the 10 ms floor almost all day; its paired difference to the best fixed Δ was -0.001 ms [-0.004, 0.002] at $\bar{\lambda} = 200$/s, $+0.005$ ms [0.001, 0.010] at 2000/s (10 seeds) and -0.83 ms [-0.91, -0.75] at 20,000/s (5 seeds, 1-h peak), so it was demoted. Work-conserving closing beat it by 4.93 and 4.41 ms at 200 and 2000/s, and fixed $\Delta \geq 3$ s violated the SLO in every 5-min bin. Second-run costs moved D by at most 0.1 ms (96.17→96.08 ms). With stapling, the optimum is simply the smallest Δ the issuer can sustain.

Simulator consistency (first-run costs). An independent event-driven simulator agrees with the Monte Carlo engine to within 0.56% on misses per token, 0.09% on D and 0.08% on issuance wait, over 45 metric rows ($\bar{\lambda} = 200$/s, $E = 100$, 1 h, 5 seeds). This checks the implementation for internal consistency only. It is not a validation.

Earlier latency campaign (P2, R1). Superseded by the R2 results in Section VI-B. At about 0.6 utilization, the added p50 relative to HMAC was -2.60 ms [-6.29, 0.55] for O1, +12.14 ms [3.17, 16.52] for B5 and +11.75 ms [2.61, 203.82] for B5v (per-run median, [min, max]); one B5v run collapsed, which we attribute, without verification, to the userspace link emulator carrying 3.3 KB URLs.

Earlier capacity campaign. A quiet P1 campaign over 12 schemes, with edge utilization ≥ 0.99 in all 60 runs, measured edge CPU per request of 20.54 µs for B1, 22.91 for O1, 22.93 for O4 (hybrid root), 37.38 for B5v, 66.24 for B5, 71.04 for Ed25519, 98.04 for ECDSA P-256 and 115.74 for the per-token hybrid.

Sensitivity. An SLH-DSA-128s root puts its 306 ms signature on the issuance path and raises D from 0.575 to 0.881 s (pull, $\Delta = 1$ s). Halving the RTTs leaves staple-first D at 0.503 s. The simulator's parameter defaults and sweep ranges are listed in Section V and in the released `sim_params.json`.

Appendix B
Hypothesis Outcomes

TABLE IX
HYPOTHESES FIXED BEFORE THE EXPERIMENTS (COMMIT `3148c65`) AND THEIR OUTCOMES.

Hypothesis	Outcome	Evidence
H1 EMBER token <512 B to $N{=}2^{20}$; $\leq$ 0.25× B5c bytes/session	partly	503 B at 2^{16}, 588 B at 2^{20}; 0.144× (h2 URL), 0.256× (HTTP/1.1); M2 smaller still
H2 verify $\leq 5\times$ HMAC and $\geq 10\times$ cheaper than ML-DSA	partly	native 7.9× / 43×; Go 3.6× / 10.7×; whole request 1.15× HMAC
H3 issuer $\geq 100\times$ per-token signing, $N \geq 2^8$	cond.	holds at $N \geq 2^8$, reached only with pooled trees
H4 root bytes $< 0.1\%$; roll-up $\geq 5\times$ fewer misses	partly	$\leq 1.04{\cdot}10^{-4}$ except per-tenant push ($5.0 \cdot 10^{-3}$); roll-up 1.4–2.3%
H5 adaptive beats best fixed Δ	refuted	$\leq$0.83 ms; demoted
H6 $< 1\%$ requests wait; p99 edge CPU < 1 ms	yes (model)	0.049% at default p_{move}, 0 with staple-all; p99 $\leq$2.61 µs (max over the second-run sweep)
H7 hybrid root $< 1\%$ edge cost	mixed	+0.09% in P1; +41.1% simulated
H8 simulator within 15% of prototype	partly	misses and bytes 24/24 rows; latency 8/24
H9 native issuer $\geq 50\times$ per-token; B11/B12 comparable	partly	192–522× at $N{=}2^8$–2^{14}; B11 issues faster than per-tenant EMBER

References

[1] Akamai Technologies, "EdgeAuth-Token-Java: Akamai edge authorization token," GitHub repository, hMAC-SHA256 token generation for cookie, query string and header tokens. Accessed 2026-11-05. [Online]. Available: https://github.com/akamai/edgeauth-token-java

[2] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[3] Amazon Web Services, "Specify signers that can create signed URLs and signed cookies (Amazon CloudFront developer guide)," Product documentation, key groups accept RSA-2048 or ECDSA-256 public keys. Accessed 2026-11-05. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-trusted-signers.html

[4] Google Cloud, "Media CDN: Generate signatures," Product documentation, ed25519-signed URLs, URL prefixes and cookies. Accessed 2026-11-05. [Online]. Available: https://cloud.google.com/media-cdn/docs/generate-signatures

[5] J. Liang, J. Jiang, H. Duan, K. Li, T. Wan, and J. Wu, "When HTTPS meets CDN: A case of authentication in delegated service," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2014, pp. 67–82.

[6] F. Cangialosi, T. Chung, D. Choffnes, D. Levin, B. M. Maggs, A. Mislove, and C. Wilson, "Measurement and analysis of private key sharing in the HTTPS ecosystem," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. ACM, 2016, pp. 628–640.

[7] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[8] B. Westerbaan, "Cloudflare targets 2029 for full post-quantum security," Cloudflare Blog, Apr. 2026. [Online]. Available: https://blog.cloudflare.com/post-quantum-roadmap

[9] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," NIST, FIPS 204, Aug. 2024.

[10] M. Prorock and O. Steele, "ML-DSA for JSON object signing and encryption (JOSE) and CBOR object signing and encryption (COSE)," RFC 9964, May 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9964

[11] R. C. Merkle, "A certified digital signature," in *Advances in Cryptology – CRYPTO '89*, ser. LNCS, vol. 435. Springer, 1990, pp. 218–238.

[12] C. Aguilar-Melchor, M. R. Albrecht, T. Bailleux, N. Bindel, J. Howe, A. Hülsing, D. Joseph, and M. Manzano, "Batch signatures, revisited," in *Topics in Cryptology – CT-RSA 2024*, ser. LNCS, vol. 14643. Springer, 2024, pp. 163–186.

[13] A. Fregly, J. Harvey, B. S. Kaliski Jr., and S. Sheth, "Merkle tree ladder mode: Reducing the size impact of NIST PQC signature algorithms in practice," in *Topics in Cryptology – CT-RSA 2023*, ser. LNCS, vol. 13871. Springer, 2023, pp. 415–441.

[14] B. Berg, D. S. Berger, S. McAllister, I. Grosof, S. Gunasekar, J. Lu, M. Uhlar, J. Carrig, N. Beckmann, M. Harchol-Balter, and G. R. Ganger, "The CacheLib caching engine: Design and experiences at scale," in *Proc. 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2020, pp. 753–768.

[15] Meta Platforms, "Evaluating SSD hardware for Facebook workloads (CacheLib documentation; anonymized production traces, CDN trace `cdn/202303`)," Product documentation and public S3 bucket `cachelib-workload-sharing`, traces licensed as CacheLib (Apache-2.0). Accessed 2026-11-06. [Online]. Available: https://github.com/facebook/CacheLib/blob/main/website/docs/Cache_Library_User_Guides/Cachebench_FB_HW_eval.md

[16] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP semantics," RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[17] Microsoft, "Maximum URL length is 2,083 characters in Internet Explorer (KB 208427)," Microsoft Support, accessed 2026-11-06. [Online]. Available: https://support.microsoft.com/kb/208427

[18] A. Barth, "HTTP state management mechanism," RFC 6265, Apr. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6265

[19] R. Peon and H. Ruellan, "HPACK: Header compression for HTTP/2," RFC 7541, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7541

[20] M. Thomson and C. Benfield, "HTTP/2," RFC 9113, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9113

[21] National Institute of Standards and Technology, "Stateless hash-based digital signature standard," NIST, FIPS 205, Aug. 2024.

[22] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," in *Network and Distributed System Security Symposium (NDSS)*, 2020.

[23] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Post-Quantum Cryptography (PQCrypto 2020)*, ser. LNCS, vol. 12100. Springer, 2020, pp. 72–91.

[24] M. Sosnowski, F. Wiedner, E. Hauser, L. Steger, D. Schoinianakis, S. Gallenmüller, and G. Carle, "The performance of post-quantum TLS 1.3," in *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies (CoNEXT 2023)*. ACM, 2023, pp. 19–27.

[25] P. Kampanakis and W. Childs-Klein, "The impact of data-heavy, post-quantum TLS 1.3 on the time-to-last-byte of real-world connections," in *NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb)*, 2024, iACR ePrint 2024/176. [Online]. Available: https://eprint.iacr.org/2024/176

[26] Cloudflare, "Cloudflare now uses post-quantum cryptography to talk to your origin server," Cloudflare Blog, Sep. 2023. [Online]. Available: https://blog.cloudflare.com/post-quantum-to-origins/

[27] Akamai, "Future-proofing the Internet: Akamai achieves end-to-end PQC," Akamai Blog, Aug. 2026. [Online]. Available: https://www.akamai.com/blog/security/2026/aug/future-proofing-internet-akamai-achieves-end-to-end-pqc

[28] Cloudflare, "Post-quantum authentication to origins is now supported," Cloudflare Blog, 2026. [Online]. Available: https://blog.cloudflare.com/post-quantum-authentication-to-origins/

[29] P. Schwabe, D. Stebila, and T. Wiggers, "Post-quantum TLS without handshake signatures," in *ACM SIGSAC Conference on Computer and Communications Security (CCS '20)*. ACM, 2020, iACR ePrint 2020/534.

[30] S. Celi, A. Faz-Hernández, N. Sullivan, G. Tamvada, L. Valenta, T. Wiggers, B. Westerbaan, and C. A. Wood, "Implementing and measuring KEMTLS," in *Progress in Cryptology – LATINCRYPT 2021*, ser. LNCS, vol. 12912. Springer, 2021, pp. 88–107.

[31] N. Bindel, U. Herath, M. McKague, and D. Stebila, "Transitioning to a quantum-resistant public key infrastructure," in *Post-Quantum Cryptography (PQCrypto 2017)*, ser. LNCS, vol. 10346. Springer, 2017, pp. 384–405.

[32] D. Benjamin, "Batch signing for TLS," IETF Internet-Draft draft-ietf-tls-batch-signing-00 (expired), Jan. 2020. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-ietf-tls-batch-signing

[33] J. Harvey *et al.*, "Merkle tree ladder (MTL) mode signatures," IETF Internet-Draft draft-harvey-cfrg-mtl-mode-09, work in progress, Mar. 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-harvey-cfrg-mtl-mode-09

[34] A. Kaizer, J. Harvey, B. S. Kaliski Jr., and S. Sheth, "ML-DSA-MTL for DNSSEC," IETF Internet-Draft draft-kaizer-dnsop-ml-dsa-mtl-dnssec-02, work in progress, Sep. 2026. [Online]. Available: https://www.ietf.org/archive/id/draft-kaizer-dnsop-ml-dsa-mtl-dnssec-02.txt

[35] A. Fregly *et al.*, "SLH-DSA-MTL for DNSSEC," IETF Internet-Draft draft-fregly-dnsop-slh-dsa-mtl-dnssec-06, work in progress, Mar. 2026. [Online]. Available: https://datatracker.ietf.org/doc/draft-fregly-dnsop-slh-dsa-mtl-dnssec/

[36] D. Benjamin, D. O'Brien, B. Westerbaan, L. Valenta, and F. Valsorda, "Merkle tree certificates," IETF Internet-Draft draft-ietf-plants-merkle-tree-certs-05, work in progress, Jul. 2026. [Online]. Available: https://www.ietf.org/archive/id/draft-ietf-plants-merkle-tree-certs-05.txt

[37] L. Valenta, C. Patton, V. Gonçalves, and B. Westerbaan, "Keeping the Internet fast and secure: introducing Merkle tree certificates," Cloudflare Blog, Oct. 2025. [Online]. Available: https://blog.cloudflare.com/bootstrap-mtc/

[38] Cloudflare, "Building a post-quantum certificate authority with Merkle tree certificates," Cloudflare Blog, 2026. [Online]. Available: https://blog.cloudflare.com/pq-ca-with-mtcs/

[39] R. Burls, "Merkle tree agent certificates (MTAC)," IETF Internet-Draft draft-burls-mtac-01, work in progress, Oct. 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-burls-mtac-01

[40] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS$^+$ signature framework," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*. ACM, 2019, pp. 2129–2146.

[41] R. Barnes, S. Iyengar, N. Sullivan, and E. Rescorla, "Delegated credentials for TLS and DTLS," RFC 9345, Jul. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9345

[42] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Network and Distributed System Security Symposium (NDSS)*, 2014.

[43] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. B. Jones, and D. Waite, "OAuth 2.0 demonstrating proof of possession (DPoP)," RFC 9449, Sep. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9449

[44] G.-V. Policharla, B. Westerbaan, A. Faz-Hernández, and C. A. Wood, "Post-quantum privacy pass via post-quantum anonymous credentials," IACR Cryptology ePrint Archive, Paper 2023/414, 2023. [Online]. Available: https://eprint.iacr.org/2023/414

[45] D. Springall, Z. Durumeric, and J. A. Halderman, "Measuring the security harm of TLS crypto shortcuts," in *Proceedings of the 2016 Internet Measurement Conference (IMC '16)*. ACM, 2016, pp. 33–47.

[46] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," RFC 9162, Dec. 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9162

[47] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," NIST, FIPS 203, Aug. 2024.

[48] D. Litinski, "How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates," arXiv:2306.08585, 2023. [Online]. Available: https://arxiv.org/abs/2306.08585